

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

MODULE 2

NHẮC LẠI KIẾN THỨC JAVA CƠ BẢN



Bùi Thị Diễm Trinh
Khoa CNTT, Đại Học Nam Cần Thơ
Email: btdtrinh@nctu.edu.vn



NỘI DUNG CHÍNH

1. Chương trình Java



2. Các khái niệm cơ bản



3. Các lệnh đơn giản



PHẦN 1. CHƯƠNG TRÌNH JAVA



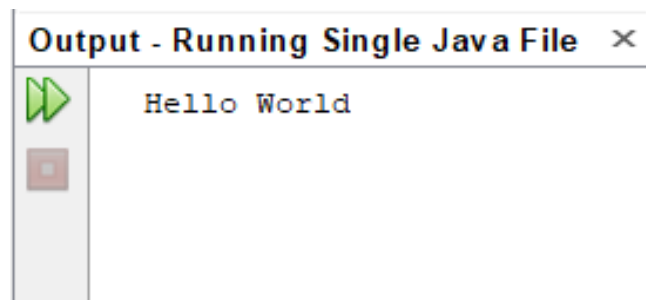


CHƯƠNG TRÌNH ĐẦU TIÊN – HELLO WORLD

- Chương trình in câu chào ra màn hình

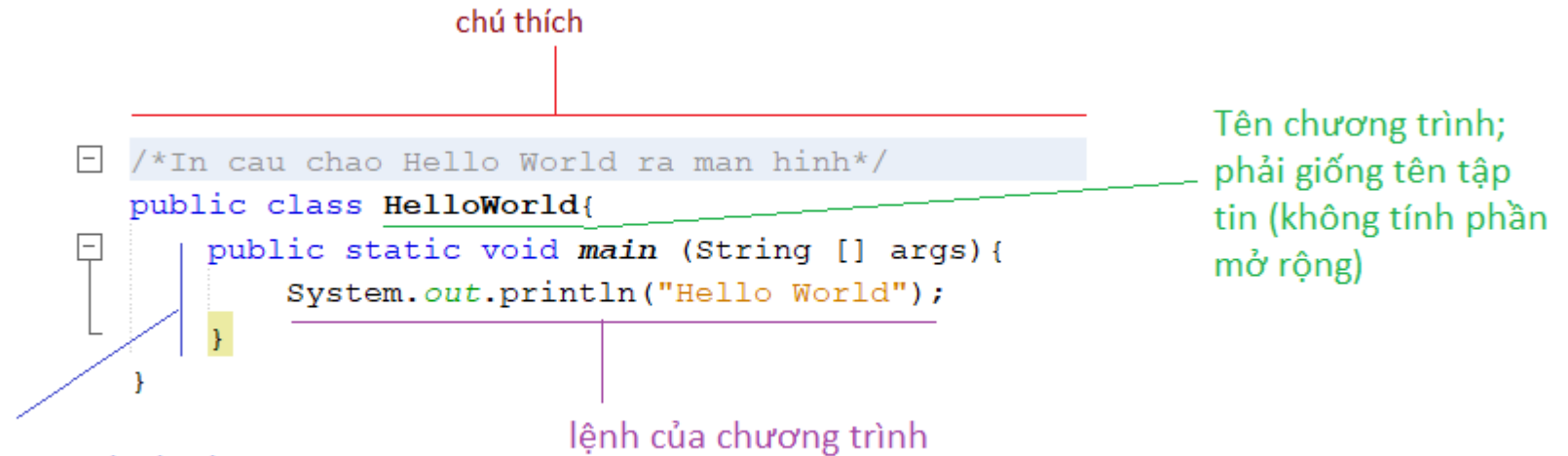
```
/*In cau chao Hello World ra man hinh*/  
public class HelloWorld{  
    public static void main (String [] args){  
        System.out.println("Hello World");  
    }  
}
```

- Kết quả thực thi chương trình





CẤU TRÚC MỘT CHƯƠNG TRÌNH JAVA



■ Chú ý

- Tên chương trình phải giống tên của tập tin (không tính phần mở rộng).
- **main()**: phương thức bắt đầu chạy
- Lớp có thể có nhiều phương thức nhưng `main()` được gọi tự động khi chương trình chạy.



PHƯƠNG THỨC MAIN

- Phương thức `main()`: là điểm bắt đầu thực thi một ứng dụng.
- Mỗi ứng dụng Java phải chứa một phương thức main có dạng như sau: `public static void main(String[] args)`
- Phương thức main chứa ba bổ từ đặc tả sau:
 - **public**: chỉ ra rằng phương thức main có thể được gọi bởi bất kỳ đối tượng nào.
 - **static**: chỉ ra rằng phương thức main là một phương thức lớp.
 - **void**: chỉ ra rằng phương thức main sẽ không trả về bất kỳ một giá trị nào.



CÂU LỆNH & CHÚ THÍCH

- Một câu lệnh kết thúc bằng dấu “;”
- Các lệnh và các định nghĩa (identifier) **phân biệt chữ hoa, chữ thường** (case sensitive)
 - `System.out.println("Hello!");` ✓
 - `System.Out.println("Hello");` ✗
- Có hai loại chú thích trong Java
 - Trên một dòng: `//chú thích`
 - Nhiều dòng: `/* chú thích */`
- Khối lệnh được bao đóng bởi cặp `{ }`



CHƯƠNG TRÌNH VỚI NHIỀU HÀM

hàm tính
tổng của 2
số nguyên

```
public class tinhTong {  
    static int sum (int a, int b){//tinh tong hai so a và b  
        return a+b;  
    }  
    public static void main(String args[]) {  
        System.out.println("Tong cua 2 so 3 và 5 la: "+sum(3, 5));  
    }  
}
```

gọi hàm tính tổng

- Kết quả thực thi chương trình

```
Output - Running Single Java File ×  
Tong cua 2 so 3 và 5 la: 8
```




CHƯƠNG TRÌNH CÓ SỬ DỤNG “LỚP” SẴN DÙNG

- Chương trình nhận vào chuỗi từ bàn phím và in ra màn hình chuỗi đó

Khai báo sử dụng lớp Scanner

```
import java.util.Scanner;
public class viDuImport {

    public static void main(String args[]) {
        System.out.print("Nhập vào một chuỗi: ");
        Scanner scanner = new Scanner(System.in);
        String inputString = scanner.nextLine();
        System.out.println("Chuỗi nhập vào là : \n"+inputString);
    }
}
```

Đọc vào một chuỗi từ bàn phím

- Lệnh **import**: khai báo sử dụng các lớp (thư viện lớp) từ bên ngoài.

```
C:\Program Files\Java\jdk-15\bin>javac viDuImport.java

C:\Program Files\Java\jdk-15\bin>java viDuImport
Nhập vào một chuỗi: Bui Thi Diem Trinh
Chuỗi nhập vào là :
Bui Thi Diem Trinh
```

PHẦN 2. CÁC KHÁI NIỆM CƠ BẢN





BIẾN

- Biến là một vùng nhớ được đặt tên, dùng để lưu trữ dữ liệu phải xử lý trong chương trình.
- Biến được đặt tên theo quy tắc đặt tên **định danh**
 - Có thể chứa các ký tự (A-Z, a-z), số(0-9), dấu _ và \$
 - Ký tự đầu tiên không được là ký tự số
 - Không được trùng với các từ khóa (keywords) của Java
 - Phân biệt chữ hoa và chữ thường

DataType **varName** [=value] [, varName2]

Kiểu dữ liệu của biến

Tên biến

Nếu giá trị của biến mà không thay đổi trong suốt quá trình hoạt động thì gọi là hằng

int = a; double b =

TỪ KHÓA (KEYWORDS)

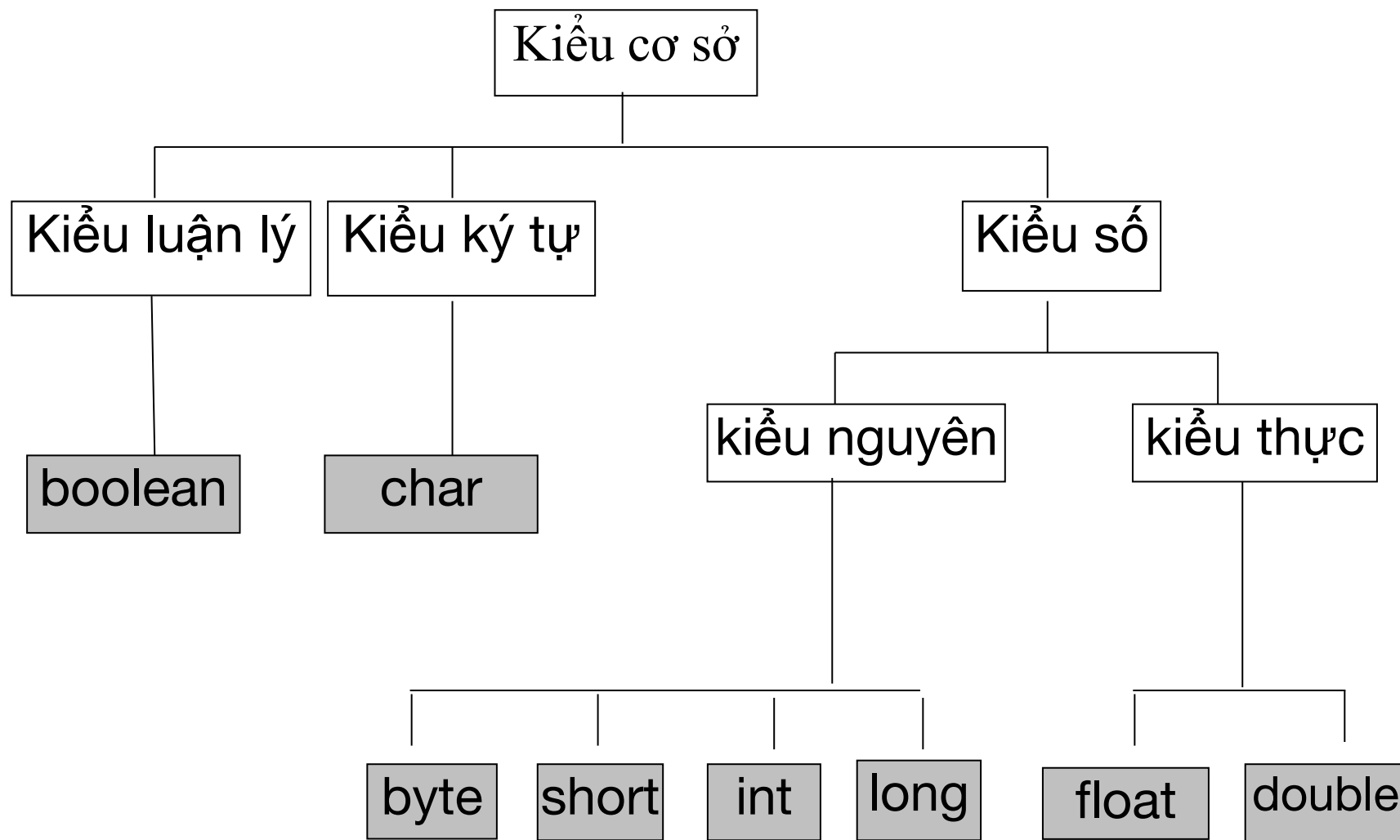
- Là các từ được Java quy định sẵn

abstarct	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while



CÁC KIỂU DỮ LIỆU NGUYÊN THỦY (PRIMITIVE DATATYPE)

- Java có 8 kiểu dữ liệu nguyên thủy (primitive datatype)





Kiểu dữ liệu số (số nguyên, số thực)

Kiểu	Kích thước	Miền giá trị
int	4 bytes	−2.147.483.648 → 2.147.483. 647
short	2 bytes	−32.768 → 32.767
long	8 bytes	−9.223.372.036.854.775.808 → 9.223.372.036.854.775.807
byte	1 byte	−128 → 127
float	4 bytes	±3.40282347E+38F
double	8 bytes	±1.79769313486231570E+308



KIỂU LUẬN LÝ

- Nhận 1 trong 2 giá trị:
 - true
 - false
- Các biến/giá trị luận lý được sử dụng như điều kiện trong các câu lệnh rẽ nhánh hoặc lặp.



Kiểu ký tự

- Kiểu ký tự cho phép thao tác trên 1 ký tự.
- Giá trị hằng kiểu ký tự được bao trong một cặp dấu ngoặc đơn “.”.
 - Ví dụ: ‘a’, ‘\n’, ‘\t’.
- Mỗi ký tự tương ứng với một mã ký tự (character code).
 - Ví dụ: ký tự ‘A’ có mã là 65, ...
- Mã ký tự có giá trị từ 0 – 65.535
- Kích thước của mỗi ký tự là 2 bytes (Unicode)

CÁC PHÉP TOÁN SỐ HỌC

Toán tử	Ý nghĩa	Ví dụ
+	Phép cộng	<code>int tong = 5 + 6;</code>
-	Phép trừ	<code>int hieu = 7 - 9;</code>
*	Phép nhân	<code>int tich = 5 * 6;</code>
/	Phép chia	<code>double = 9 / 4.5;</code>
%	Phép chia lấy phần dư	<code>int phanDu = 7 % 5;</code>
++	Phép tăng 1 đơn vị	<code>a++; ++a;</code>
--	Phép giảm 1 đơn vị	<code>a--; --a;</code>

Độ ưu tiên: 1- Nhân và chia; 2- Cộng và trừ; 3- Trái sang phải

CÁC PHÉP TOÁN SO SÁNH

- Toán tử so sánh (comparisons):

`==` `!=` `<` `<=` `>` `>=`

- Toán tử luận lý (boolean operators):

`&&` `||` `!`

- Toán tử kết hợp (combined operators): kết hợp các toán tử toán học và phép gán.

`+=` `-=` `*=` `/=` `%=`



BIỂU THỨC

- Là một biến hay sự kết hợp giữa toán tử với các biến, hằng hoặc các biểu thức khác.
 - `3 + b * 5` // Biểu thức toán học
 - `(year % 4) == 0` // Biểu thức luận lý
- Quy tắc chuyển đổi kiểu của biểu thức toán học:
 - Nếu tất cả các toán hạng là kiểu nguyên thì biểu thức trả về kiểu nguyên.
 - Nếu có ít nhất một toán hạng kiểu số thực thì biểu thức trả về kiểu thực.

ÉP KIỂU (TYPE CASTING)

- Là ép kiểu dữ liệu của một giá trị từ kiểu này sang kiểu khác.
- Ép kiểu tự động: Java tự động ép kiểu khi có sự không tương thích về kiểu.
- Ép kiểu tường minh: người lập trình thực hiện ép kiểu một cách tường minh.

(float) 5 / 2 + 2.0 = 2.5 + 2.0 = 4.5



CHUYỂN ĐỔI KIỂU

- Java cung cấp một số hàm để chuyển đổi kiểu.
- Các hàm chuyển đổi kiểu thông dụng:
 - **int Integer.parseInt (String s)**: trả về giá trị số nguyên tương ứng với một chuỗi số.
 - **float Float.parseFloat (String s)**: trả về giá trị số thực (float) tương ứng với một chuỗi số.
 - **double Double.parseDouble (String s)**: trả về giá trị số thực (double) tương ứng với một chuỗi số.
 - **String.valueOf (a)**: trả về giá trị chuỗi của biến a.
 - ...



Kiểu chuỗi

- Được xem như là một xâu các ký tự.
- Một hằng chuỗi được bao trong cặp ngoặc kép "".
 - Ví dụ: "Hôm nay là ngày đầu tuần. "
- Mỗi ký tự trong chuỗi có tương ứng với một chỉ số, chỉ số của ký tự đầu tiên của chuỗi được đánh số là 0.
- Kiểu dữ liệu chuỗi trong Java: String
- Toán tử trên chuỗi: + (cộng, ghép chuỗi)
 - `System.out.println("Gia tri cua bien a la: "+a);`



XUẤT DỮ LIỆU RA MÀN HÌNH

- `System.out.println(String s)`: xuất dữ liệu chuỗi và xuống dòng.
- `System.out.print(String s)`: xuất dữ liệu chuỗi và không xuống dòng.
- `System.out.printf(String format, Object... args)`: xuất dữ liệu có định dạng.
- Xem chi tiết cú pháp tại:
<https://docs.oracle.com/javase/8/docs/api/java/util/Formatter.html#syntax>



ĐỌC DỮ LIỆU TỪ BÀN PHÍM

- Để đọc dữ liệu từ bàn phím sử dụng kết hợp lớp **Scanner** và **System.in**
- Tạo một đối tượng thuộc lớp Scanner:
`Scanner input = new Scanner (System.in);`
- Lớp Scanner được đóng gói trong **java.util**, nên ta phải thêm vào chương trình bằng lệnh: **import java.util.Scanner;**
- Các phương thức đọc dữ liệu
 - **String nextLine()**: đọc một chuỗi ký tự
 - **int nextInt()**: đọc một số nguyên kiểu int; **long nextLong()**: đọc một số nguyên kiểu long
 - **float nextFloat()**: đọc một số thực kiểu float; ...



ĐỌC DỮ LIỆU TỪ BÀN PHÍM

- Nhập số nguyên

```
System.out.println("Nhập số nguyên: ");  
int a=sc.nextInt();
```

- Nhập số thực, double

```
System.out.println("Nhập số thực: ");  
float b=sc.nextFloat();  
System.out.println("Nhập số double: ");  
double c = sc.nextDouble();
```

- Nhập chuỗi

```
System.out.println("Nhập tên: ");  
String ten = sc.nextLine();
```

- Nhập chuỗi và số hoặc chuỗi liên tục

```
String s=new Scanner(System.in).nextLine();  
int x =new Scanner(System.in).nextInt();
```




VÍ DỤ NHẬP/XUẤT DỮ LIỆU ĐƠN GIẢN

- Nhập dữ liệu, sử dụng đối tượng **Scanner** trong thư viện **java.util.***

Viết chương trình: Nhập xuất các thông tin của 1 sinh viên gồm: Mã sinh viên, họ tên, năm sinh, điểm lý thuyết, điểm thực hành. Xuất ra các thông tin đã nhập và điểm trung bình

```
Problems Javadoc Declaration Console
<terminated> bai01 [Java Application] C:\Program Files\Java\jre1.8.0_181\bin\javaw.exe
NHẬP/XUẤT THÔNG TIN CỦA SINH VIÊN
Nhập mã số: 123
Nhập họ tên: Bùi Thị Diễm Trinh
Nhập năm sinh: 1989
Điểm lý thuyết: 8
Điểm thực hành: 8.5
-----THÔNG TIN HIỂN THỊ-----
123; Bùi Thị Diễm Trinh; 1989; ĐTB = 8.25
```




VÍ DỤ NHẬP/XUẤT DỮ LIỆU ĐƠN GIẢN

- **Đ**ọc vào 2 số nguyên và **i**n ra màn hình tổng của 2 số

```
import java.util.Scanner;
public class viDuNhapXuatDonGian {

    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);
        System.out.println("Nhap vao gia tri nguyen cho bien a: ");
        int a = input.nextInt();
        System.out.println("Nhap vao gia tri nguyen cho bien b: ");
        int b = input.nextInt();
        System.out.println("Tong 2 so nguyen a và b là: " + (a+b));
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuNhapXuatDonGian
Nhap vao gia tri nguyen cho bien a:
6
Nhap vao gia tri nguyen cho bien b:
9
Tong 2 so nguyen a và b là: 15
```



VÍ DỤ NHẬP/XUẤT DỮ LIỆU ĐƠN GIẢN

- **Đ**ọc vào 2 số nguyên và **i**n ra màn hình tổng của 2 số nguyên đó.

viDuNhapXuatDonGian.class	01/01/2023 11:58 PM	CLASS File	2 KB
viDuNhapXuatDonGian	01/01/2023 11:54 PM	JAVA File	1 KB

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.22000.1335]
(c) Microsoft Corporation. All rights reserved.

C:\GiangDay\LapTrinhJava\Vidu_Java>javac viDuNhapXuatDonGian.java

C:\GiangDay\LapTrinhJava\Vidu_Java>java viDuNhapXuatDonGian
Nhap vao gia tri nguyen cho bien a:
12
Nhap vao gia tri nguyen cho bien b:
3
Tong 2 so nguyen a va b la: 15

C:\GiangDay\LapTrinhJava\Vidu_Java>
```



BÀI TẬP

- Cho 2 số tự nhiên $a = 79$ và $b = 28$ áp dụng các hàm cơ bản. Thực hiện các phép tính: So sánh Min/max, bình phương, lập phương, căn bậc hai 1 số, làm tròn số và tạo số ngẫu nhiên.
- Lưu ý: Thực thi chương trình bằng 2 cách: Thực thi bằng công cụ soạn thảo và thực thi bằng Command Line

```
min(79, 28) = 28
max(79, 28) = 79
79^2= 6241.0
28^3= 21952.0
Can bac 2 cua 79= 8.888194417315589
round - Can bac 2 cua 79= 9
floor - Can bac 2 cua 79= 8.0
ceil - Can bac 2 cua 79= 9.0
Phat sinh so ngau nhien = 0.4014240294881507
Tri tuyet doi cua 28-79=51
```



HƯỚNG DẪN

- Để thực hiện được các phép tính trên cần sử dụng đối tượng Math.
 - `Math.min(số thứ nhất, số thứ hai)`: So sánh tìm ra số nhỏ nhất trong 2 số.
 - `Math.pow(Số, số mũ)`: Tính giá trị mũ.
 - `Math.sqrt(số)`: Lấy căn bậc hai
 - `Math.round(số)/floor(số)/ceil(số)`: Làm tròn số
 - `Math.random()`: Lấy số ngẫu nhiên, số ngẫu nhiên này nhỏ hơn 1.
 - ...



```
package Buoi1;
```

```
public class BaiTap3 {  
    public static void main(String[] arg) {  
        int a = 79;  
        int b = 28;  
        System.out.println("min("+a+", "+b+") = " +  
Math.min(a, b));  
        System.out.println("max("+a+", "+b+") = " +  
Math.max(a, b));  
        System.out.println(a+"^2= " + Math.pow(a, 2));  
        System.out.println(b+"^3= " + Math.pow(b, 3));  
        System.out.println("Can bac 2 cua " +a+ "= " +  
Math.sqrt(a));  
        System.out.println("round - Can bac 2 cua " +a+ "= "  
+ Math.round(Math.sqrt(a)));  
        System.out.println("floor - Can bac 2 cua " +a+ "= "  
+ Math.floor(Math.sqrt(a)));  
        System.out.println("ceil - Can bac 2 cua " +a+ "= "  
+ Math.ceil(Math.sqrt(a)));  
        System.out.println("Phat sinh so ngau nhien = " +  
Math.random());  
        System.out.println("Tri tuyet doi cua " +b+ "-" +a+  
"=" +Math.abs(a-b));  
    }  
}
```



PHẦN 3. CÁC LỆNH ĐƠN GIẢN





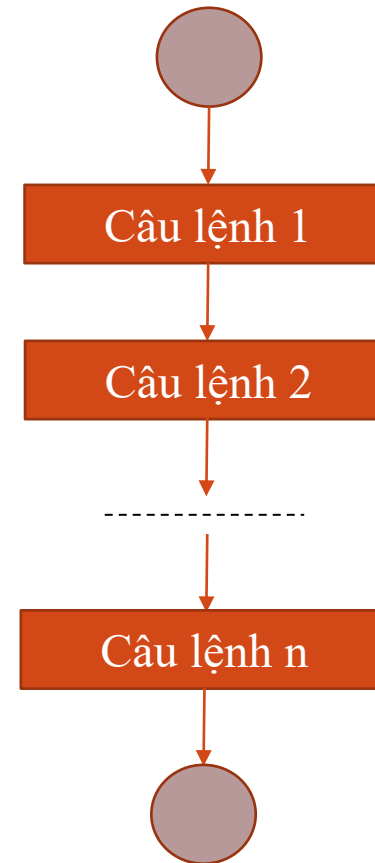
CẤU TRÚC ĐIỀU KHIỂN TRONG JAVA

- Điều khiển cách thực thi các câu lệnh trong chương trình.
- Java có 3 cấu trúc điều khiển:
 - Tuần tự
 - Rẽ nhánh
 - Lặp



CẤU TRÚC TUẦN TỰ

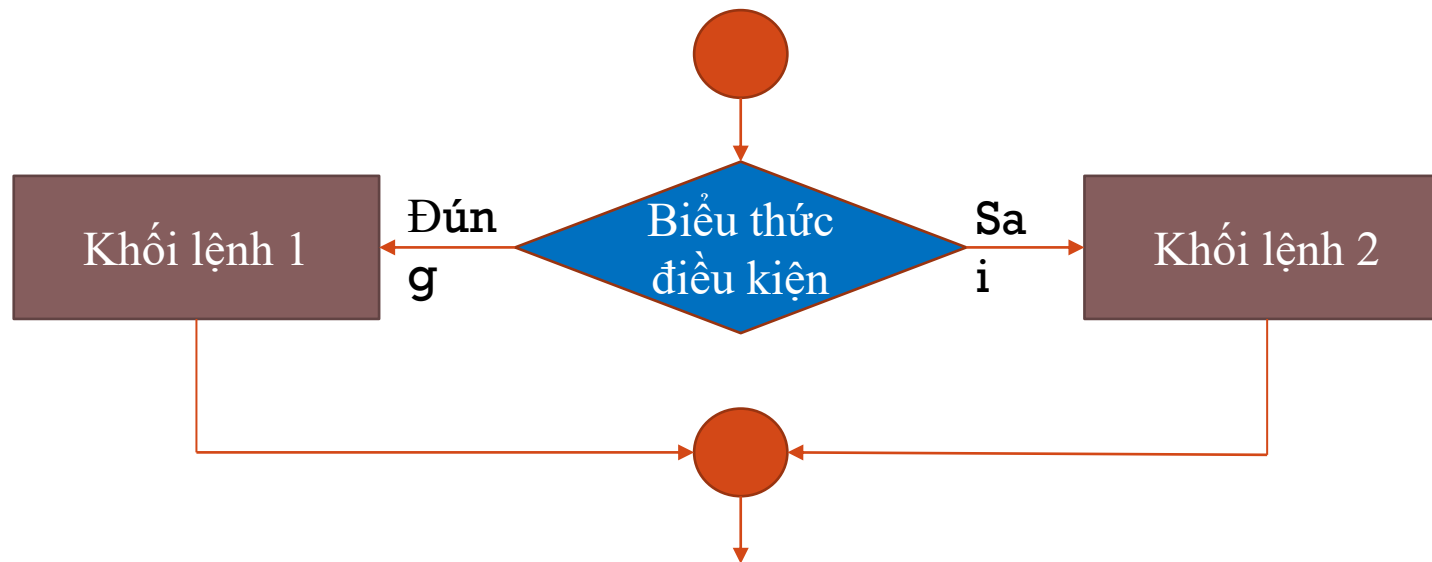
- Các câu lệnh thực hiện tuần tự nhau theo thứ tự trước sau.
- Là cấu trúc điều khiển mặc định.





CẤU TRÚC RẺ NHÁNH – LỆNH IF .. ELSE

- Lựa chọn thực hiện 1 trong 2 công việc (1 trong 2 khối lệnh) dựa vào giá trị luận lý của biểu thức điều kiện.
- Các lệnh if .. else có thể sử dụng lồng nhau.





CẤU TRÚC RẺ NHÁNH – LỆNH IF .. ELSE

- Lệnh if đầy đủ:

```
if (biểu thức điều kiện){  
    Khối lệnh đúng  
}  
else{  
    Khối lệnh sai  
}
```
- Nếu điều kiện đúng, thực hiện khối lệnh đúng, ngược lại thực hiện khối lệnh sai.
 - Trong trường hợp khối lệnh chỉ có 1 câu lệnh thì không cần cặp dấu ngoặc nhọn.



CẤU TRÚC RẺ NHÁNH – LỆNH IF .. ELSE

- Ví dụ: Số chẵn – lẻ

```
if(i%2==0)
    {System.out.println("i la so chan");}
else
    {System.out.println("i la so le");}
```

- Ví dụ:

```
if((y%4==0 && y%100!=0) || y%400==0)
    {System.out.println("y la nam nhuan");}
else
    {System.out.println("y khong la nam nhuan");}
```



CẤU TRÚC RẺ NHÁNH – LỆNH IF .. ELSE

- Chương trình nhập vào 2 số nguyên a và b, nếu $b = 0$ thì báo lỗi không thể thực hiện phép chia, ngược lại in ra màn hình kết quả phép toán a/b .

```
import java.util.Scanner;
public class viDuReNhanhIfPhepChia {

    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);
        System.out.println("Nhap vao gia tri nguyen cho bien a: ");
        int a = input.nextInt();
        System.out.println("Nhap vao gia tri nguyen cho bien b: ");
        int b = input.nextInt();
        if(b == 0){
            System.out.println("b = 0, khong the thuc hien phep chia!");
        }
        else{
            System.out.println("Ket qua phep tinh a/b la: "+((float)a/b));
        }
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuReNhanhIfPhepChia
Nhap vao gia tri nguyen cho bien a:
5
Nhap vao gia tri nguyen cho bien b:
0
b = 0, khong the thuc hien phep chia!
```

```
C:\GiangDay\ViDu_Java>java viDuReNhanhIfPhepChia
Nhap vao gia tri nguyen cho bien a:
3
Nhap vao gia tri nguyen cho bien b:
6
Ket qua phep tinh a/b la: 0.5
```




CẤU TRÚC LỰA CHỌN – LỆNH SWITCH .. CASE

switch (biểu thức) {

case x:

// khối lệnh

break;

case y:

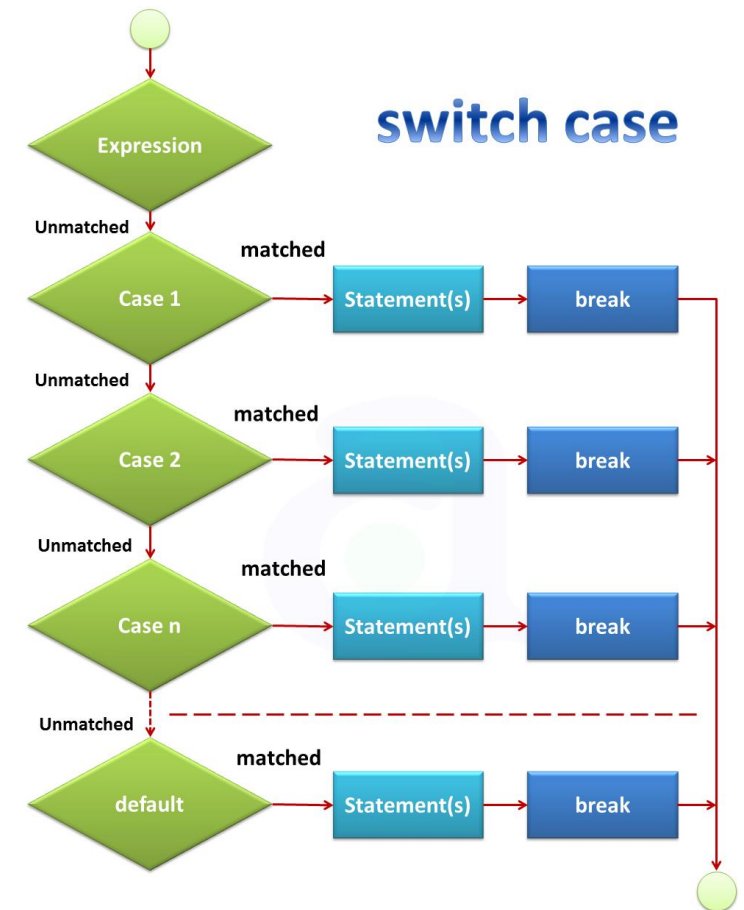
// khối lệnh

break;

...

default: *//không bắt buộc phải có*
// khối lệnh

}





CẤU TRÚC LỰA CHỌN – LỆNH SWITCH .. CASE

- “*Biểu thức*” phải là một biểu thức có giá trị kiểu số nguyên, kiểu ký tự hoặc kiểu chuỗi.
- Mệnh đề **case** nào có giá trị bằng với giá trị của “*Biểu thức*”, thì các câu lệnh từ mệnh đề **case** đó cho đến hết lệnh **switch** sẽ được thực hiện, hoặc cho đến khi gặp lệnh **break**.
- Lệnh **break** dùng để thoát ra khỏi cấu trúc **switch**.
- Các câu lệnh trong mệnh đề **default** (*không bắt buộc*) sẽ được thực hiện nếu giá trị của “*Biểu thức*” không nằm trong các giá trị được liệt kê trong các mệnh đề case.



CẤU TRÚC LỰA CHỌN – LỆNH SWITCH .. CASE

```
import java.util.Scanner;
public class viDuSwitch {

    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);

        System.out.println("Nhap vao gia tri thuc cho bien a: ");
        double a = input.nextDouble();
        System.out.println("Nhap vao gia tri thuc cho bien b: ");
        double b = input.nextDouble();
        System.out.println("Nhap vao phep toan: ");
        String op = input.next();
        switch(op) {
            case "+":
                System.out.println("Ket qua a "+op+" b la: "+(a+b));
                break;
            case "-":
                System.out.println("Ket qua a "+op+" b la: "+(a-b));
                break;
            case "*":
                System.out.println("Ket qua a "+op+" b la: "+(a*b));
                break;
            case "/":
                System.out.println("Ket qua a "+op+" b la: "+(a/b));
                break;
        }
    }
}
```

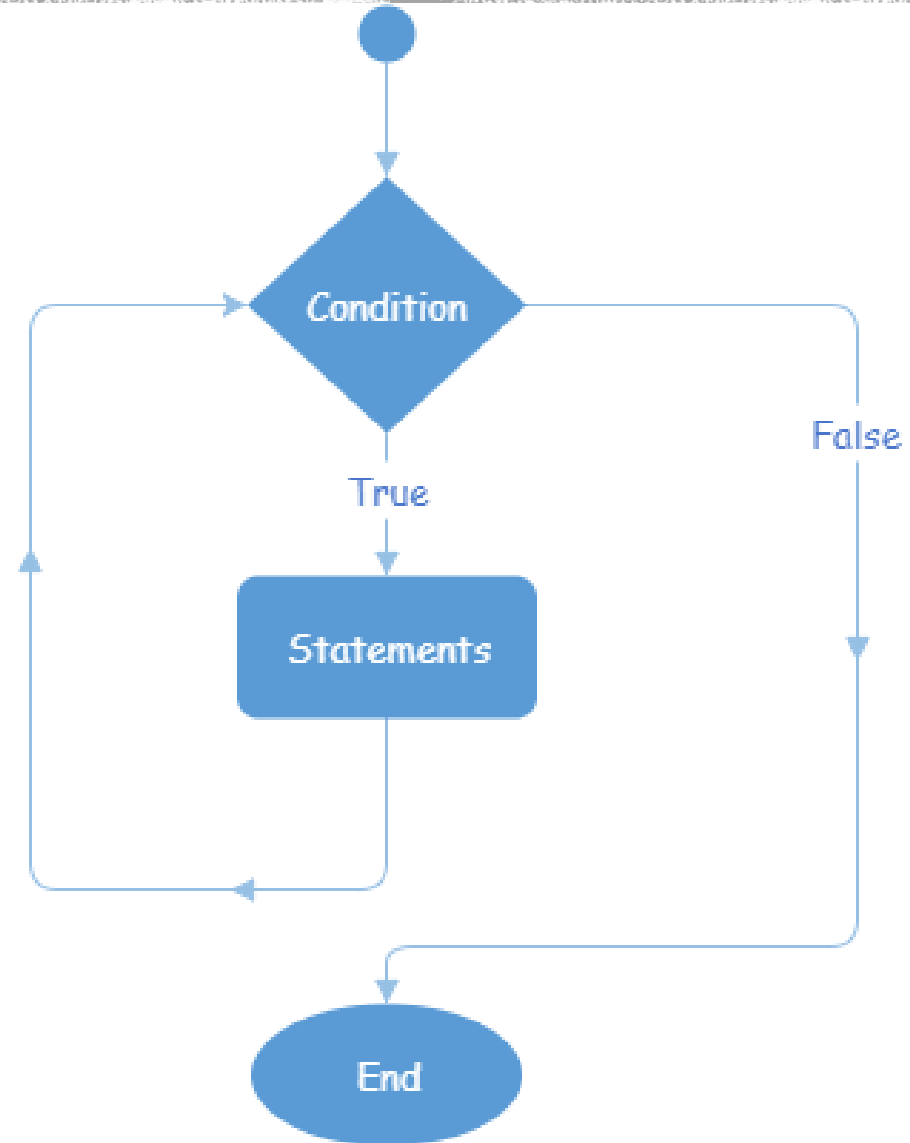
- Nhập vào 2 số thực a và b cùng 1 phép toán số học (+, -, *, /). In ra màn hình kết quả của phép toán.

```
C:\GiangDay\ViDu_Java>java viDuSwitch
Nhap vao gia tri thuc cho bien a:
6
Nhap vao gia tri thuc cho bien b:
9
Nhap vao phep toan:
+
Ket qua a + b la: 15.0
```



CẤU TRÚC LẶP

- Thực hiện lặp đi lặp lại một tác vụ (khối lệnh) một số lần dựa vào điều kiện định trước.
- Các lệnh lặp
 - while
 - do .. While
 - for





LỆNH WHILE

```
while (biểu thức điều kiện) {  
    // khối lệnh  
}
```

▪ Cách thực hiện

- Kiểm tra biểu thức điều kiện
- Nếu biểu thức điều kiện “đúng” thực hiện khối lệnh. Nếu sai thoát khỏi vòng lặp.
- Quay lại bước 1.



LỆNH WHILE

- Tính giá trị biểu thức 2^n , trong đó n nhập từ bàn phím.

```
import java.util.Scanner;
public class viDuWhile {

    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);
        System.out.println("Nhap vao gia tri nguyen cho bien n: ");
        int n = input.nextInt();
        int i=1;
        int kq = 1;
        while(i <= n){
            kq = kq * 2;
            i++;
        }
        System.out.println("Gia tri 2^"+n+" la: "+kq);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuWhile
Nhap vao gia tri nguyen cho bien n:
3
Gia tri 2^3 la: 8
```




LỆNH DO .. WHILE

do {

// khối lệnh

}

while (biểu thức điều kiện);

- Cách thực hiện
 - Thực hiện thân vòng lặp
 - Định giá trị “biểu thức điều kiện”:
 - Nếu đúng, trở lại bước 1
 - Ngược lại, thoát khỏi vòng lặp



LỆNH DO .. WHILE

- Tính giá trị biểu thức 2^n trong đó n nhận từ bàn phím.

```
import java.util.Scanner;

public class viDuDoWhile {

    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);
        System.out.println("Nhap vao gia tri nguyen cho bien n: ");
        int n = input.nextInt();
        int i=1;
        int kq = 1;
        do{
            kq = kq * 2;
            i++;
        }
        while(i <= n);
        System.out.println("Gia tri 2^"+n+" la: "+kq);
    }
}
```

```
C:\GiangDay\ViDu_Java>java viDuDoWhile
Nhap vao gia tri nguyen cho bien n:
3
Gia tri 2^3 la: 8
```



LỆNH FOR

```
for (Biểu thức 1; Biểu thức 2; Biểu thức 3) {  
    // Khối lệnh  
}
```

▪ Cách thực hiện

1. Thực hiện biểu thức 1 (còn gọi là biểu thức khởi tạo)
2. Định giá trị biểu thức 2 (còn gọi là biểu thức điều kiện)
 - ✓ Nếu biểu thức điều kiện đúng, thực hiện khối lệnh.
 - ✓ Nếu biểu thức điều kiện sai, thoát khỏi vòng lặp.
3. Thực hiện biểu thức 3 (còn gọi là biểu thức tăng)
4. Quay lại bước 2



LỆNH FOR

- Tính giá trị biểu thức 2^n , trong đó n nhập từ bàn phím.

```
import java.util.Scanner;
```

```
public class viDuFor {
```

```
    public static void main(String args[]) {
```

```
        Scanner input = new Scanner(System.in);
```

```
        System.out.println("Nhap vao gia tri nguyen cho bien n: ");
```

```
        int n = input.nextInt();
```

```
        int kq = 1;
```

```
        for(int i =0; i< n; i++){
```

```
            kq = kq * 2;
```

```
        }
```

```
        System.out.println("Gia tri 2^"+n+" la: "+kq);
```

```
    }
```

```
}
```

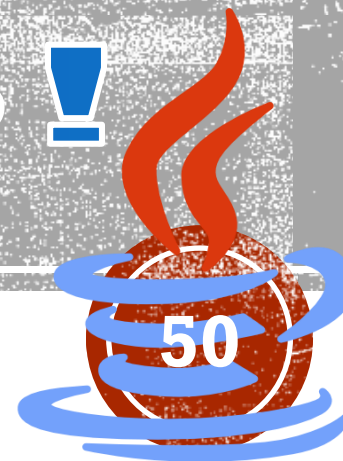
```
C:\GiangDay\ViDu_Java>java viDuFor
Nhap vao gia tri nguyen cho bien n:
3
Gia tri 2^3 la: 8
```



LỆNH BREAK, CONTINUE TRONG CẤU TRÚC LẶP

- **Lệnh break:** Kết thúc ngay một vòng lặp
 - Khi gặp **break**, vòng lặp sẽ kết thúc ngay lập tức, bất kể giá trị của biểu thức điều kiện.
 - Lệnh này cũng được sử dụng trong câu lệnh switch...case.
- **Lệnh continue:** Bắt đầu chu kỳ lặp mới
 - Khi gặp **continue**, chương trình sẽ thực hiện ngay chu kỳ lặp mới.
 - Các lệnh dưới sau continue sẽ bị bỏ qua.

Stop!





BÀI TẬP ỨNG DỤNG

- Bài 1. Viết chương trình yêu cầu người dùng nhập vào số n ($n > 0$), kiểm tra n có phải là số nguyên tố hay không?
- Bài 2. Viết chương trình tìm ước số chung lớn nhất của hai số nguyên dương a, b được nhập từ bàn phím.
- Bài 3. Phân tích số nguyên thành số nguyên tố. Ví dụ: Số 28 được phân tích thành $2 * 2 * 7$
- Bài 4. Viết chương trình liệt kê n số nguyên tố đầu tiên
- Bài 5. Dãy Fibonacci được định nghĩa như sau: $F_0 = 1, F_1 = 1, F_n = F_{n-1} + F_{n-2}$ với $n \geq 2$. Hãy viết chương trình tìm số Fibonacci thứ n .